

PLAN D'ARCHITECTURE COMPLET

Fladder (Flutter) → RYOKY-LINK (Vue 3 + TypeScript)

STRUCTURE DES DOSSIERS

```
ryoky-link/
├─ src/
│  ├─ types/                # Interfaces TypeScript (ex-models/)
│  │  ├─ jellyfin.ts        # Types API Jellyfin
│  │  ├─ item.ts            # ItemBaseModel, MediaItem, etc.
│  │  ├─ user.ts            # User, Account, Credentials
│  │  ├─ playback.ts        # Playback, MediaStreams, Segments
│  │  ├─ settings.ts        # Settings, Preferences
│  │  ├─ library.ts         # Library, Collection, View
│  │  └─ index.ts           # Re-exports
│  │
│  ├─ stores/               # Pinia stores (ex-providers/)
│  │  ├─ auth.ts            # Auth, login, logout, token
│  │  ├─ user.ts            # User info, settings, profiles
│  │  ├─ jellyfin.ts        # Service API Jellyfin (ex-service_provider)
│  │  ├─ dashboard.ts       # Dashboard data (resume, nextUp, views)
│  │  ├─ library.ts         # Library browsing, filters, search
│  │  ├─ playback.ts        # Player state, controls, media
│  │  ├─ settings.ts        # App settings, client settings
│  │  ├─ views.ts           # Views / libraries list
│  │  ├─ search.ts          # Global search
│  │  ├─ favourites.ts      # Favourites / likes
│  │  ├─ connectivity.ts    # Online/offline state
│  │  └─ index.ts           # Store initialization
│  │
│  ├─ services/             # Services métier purs
│  │  ├─ jellyfin-api.ts    # HTTP client (axios/fetch)
│  │  ├─ image-service.ts   # URL generation for posters/backdrops
│  │  ├─ playback-service.ts # Player logic, transcoding
│  │  ├─ sync-service.ts    # Offline sync (IndexedDB)
│  │  └─ local-storage.ts   # localStorage wrapper
│  │
│  ├─ composables/          # Composables Vue 3 (logique réutilisable)
│  │  ├─ useJellyfin.ts     # Accès API générique
│  │  ├─ usePlayback.ts     # Contrôles player
│  │  ├─ useImageUrl.ts     # Génération URLs images
│  │  ├─ useResponsive.ts   # Breakpoints mobile/tablet/desktop
│  │  ├─ useKeyboardShortcuts.ts # Raccourcis clavier player
│  │  ├─ useMediaSegments.ts # Skip intro/credits
│  │  └─ useFullscreen.ts   # Fullscreen API
│  │
│  └─ components/           # Composants Vue
│     └─ layout/
```

```

| | | └─ AppLayout.vue      # Layout principal (sidebar + content)
| | | └─ Sidebar.vue       # Navigation latérale
| | | └─ TopBar.vue        # Barre supérieure
| | | └─ BottomNav.vue     # Navigation mobile
| | |
| | | └─ media/
| | | | └─ PosterCard.vue  # Carte poster (film/série)
| | | | └─ PosterRow.vue   # Ligne de posters scrollable
| | | | └─ PosterGrid.vue  # Grille de posters
| | | | └─ MediaBanner.vue  # Bannière hero (dashboard top)
| | | | └─ MediaInfo.vue   # Info média (titre, année, rating)
| | | | └─ ProgressBar.vue # Barre de progression visionnage
| | | | └─ EpisodeCard.vue # Carte épisode
| | | | └─ SeasonList.vue  # Liste des saisons
| | |
| | | └─ player/
| | | | └─ VideoPlayer.vue  # Composant player principal
| | | | └─ PlayerControls.vue # Contrôles overlay
| | | | └─ PlayerOverlay.vue # Overlay (titre, boutons)
| | | | └─ ProgressBar.vue  # Barre de progression player
| | | | └─ VolumeControl.vue # Contrôle volume
| | | | └─ QualitySelector.vue # Sélecteur qualité
| | | | └─ SubtitleSelector.vue # Sélecteur sous-titres
| | | | └─ AudioSelector.vue # Sélecteur piste audio
| | | | └─ SkipSegment.vue  # Bouton skip intro/credits
| | | | └─ KeyboardHandler.vue # Handler raccourcis
| | |
| | | └─ detail/
| | | | └─ ItemDetail.vue   # Page détail item
| | | | └─ CastList.vue     # Liste acteurs
| | | | └─ RelatedItems.vue # Items similaires
| | | | └─ Overview.vue     # Synopsis
| | | | └─ ActionButtons.vue # Fav, vu, liste, etc.
| | |
| | | └─ library/
| | | | └─ LibraryView.vue  # Vue bibliothèque
| | | | └─ FilterBar.vue    # Barre filtres
| | | | └─ SortSelector.vue # Sélecteur tri
| | | | └─ ViewToggle.vue   # Toggle grille/liste
| | |
| | | └─ search/
| | | | └─ SearchBar.vue    # Barre recherche
| | | | └─ SearchResults.vue # Résultats recherche
| | | | └─ SearchFilters.vue # Filtres recherche
| | |
| | | └─ settings/
| | | | └─ SettingsPanel.vue # Panel settings
| | | | └─ PlayerSettings.vue # Settings player
| | | | └─ AppearanceSettings.vue # Settings apparence
| | | | └─ ServerSettings.vue # Settings serveur

```

```

| | | └─ ui/ # Composants UI génériques
| | |   └─ Button.vue
| | |   └─ Modal.vue
| | |   └─ LoadingSpinner.vue
| | |   └─ EmptyState.vue
| | |   └─ ErrorBoundary.vue
| | |
| | └─ views/ # Pages (routées)
| |   └─ LoginView.vue # Connexion
| |   └─ DashboardView.vue # Accueil
| |   └─ LibraryView.vue # Bibliothèque
| |   └─ ItemDetailView.vue # Détail item
| |   └─ PlayerView.vue # Lecteur plein écran
| |   └─ SearchView.vue # Recherche
| |   └─ FavouritesView.vue # Favoris
| |   └─ SettingsView.vue # Paramètres
| |   └─ LiveTvView.vue # TV en direct
| |
| └─ router/
|   └─ index.ts # Vue Router config
|
| └─ assets/ # Fonts, images, icons
| └─ styles/ # CSS global, variables, themes
| └─ utils/ # Fonctions utilitaires
| └─ App.vue # Root component
| └─ main.ts # Entry point
|
└─ public/
└─ index.html
└─ package.json
└─ vite.config.ts
└─ tsconfig.json
└─ ...

```

🔗 MAPPING FLADDER → VUE 3

1. STATE MANAGEMENT (Riverpod → Pinia)

Fladder (Riverpod)	RYOKY (Pinia)	Description
auth_provider	stores/auth.ts	Login, token, session
user_provider	stores/user.ts	User info, settings
dashboard_provider	stores/dashboard.ts	Resume, nextUp, recentlyAdded
views_provider	stores/views.ts	Liste des libraries
service_provider	stores/jellyfin.ts + services/ jellyfin-api.ts	Calls API
video_player_provider	stores/playback.ts	État player
settings/*_provider	stores/settings.ts	Tous les settings

Table 1 – continued

Fladder (Riverpod)	RYOKY (Pinia)	Description
library_screen_provider	stores/library.ts	Browse library
search_provider	stores/search.ts	Recherche
favourites_provider	stores/favourites.ts	Favoris
connectivity_provider	stores/connectivity.ts	Online/offline

2. NAVIGATION (auto_route → Vue Router)

```
// router/index.ts
const routes = [
  { path: '/login', component: LoginView, name: 'login' },
  {
    path: '/',
    component: AppLayout,
    children: [
      { path: '', component: DashboardView, name: 'dashboard' },
      { path: 'library/:id', component: LibraryView, name: 'library' },
      { path: 'item/:id', component: ItemDetailView, name: 'item' },
      { path: 'search', component: SearchView, name: 'search' },
      { path: 'favourites', component: FavouritesView, name: 'favourites' },
      { path: 'settings', component: SettingsView, name: 'settings' },
      { path: 'livetv', component: LiveTvView, name: 'livetv' },
    ]
  },
  { path: '/player', component: PlayerView, name: 'player' }, // Fullscreen
]
```

3. MODELS (Freezed → TypeScript Interfaces)

```
// types/item.ts
export interface BaseItemDto {
  id: string;
  name: string;
  type: 'Movie' | 'Series' | 'Season' | 'Episode' | 'BoxSet' | 'MusicAlbum' | 'Audio' | 'Book';
  overview?: string;
  year?: number;
  premiereDate?: string;
  officialRating?: string;
  communityRating?: number;
  criticRating?: number;
  runTimeTicks?: number;
  productionYear?: number;
  indexNumber?: number; // Episode number
  parentIndexNumber?: number; // Season number
  seriesName?: string;
  seasonName?: string;
  imageTags?: Record<string, string>;
  backdropImageTags?: string[];
```

```

    primaryImageAspectRatio?: number;
    userData?: UserItemDataDto;
    mediaType?: 'Video' | 'Audio';
    collectionType?: string;
    // ... etc
}

export interface UserItemDataDto {
    playbackPositionTicks?: number;
    playCount?: number;
    isFavorite?: boolean;
    likes?: boolean;
    lastPlayedDate?: string;
    played?: boolean;
    key?: string;
    itemId?: string;
}

export interface MediaStream {
    index: number;
    type: 'Video' | 'Audio' | 'Subtitle';
    codec?: string;
    language?: string;
    displayTitle?: string;
    isDefault?: boolean;
    isExternal?: boolean;
    path?: string;
    // ... etc
}

export interface MediaSourceInfo {
    id: string;
    protocol?: string;
    path?: string;
    type?: string;
    container?: string;
    size?: number;
    name?: string;
    isRemote?: boolean;
    hasMixedProtocols?: boolean;
    runTimeTicks?: number;
    supportsTranscoding?: boolean;
    supportsDirectStream?: boolean;
    supportsDirectPlay?: boolean;
    isInfiniteStream?: boolean;
    requiresOpening?: boolean;
    requiresClosing?: boolean;
    requiresLooping?: boolean;
    supportsProbing?: boolean;
    videoType?: string;
    isoType?: string;
}

```

```

    video3DFormat?: string;
    mediaStreams?: MediaStream[];
    mediaAttachments?: any[];
    formats?: string[];
    bitrate?: number;
    timestamp?: string;
    requiredHttpHeaders?: Record<string, string>;
    transcodingUrl?: string;
    transcodingSubProtocol?: string;
    transcodingContainer?: string;
    analyzeDurationMs?: number;
    readAtNativeFramerate?: boolean;
    itemId?: string;
}

export interface MediaSegment {
    id: string;
    itemId: string;
    type: 'Intro' | 'Outro' | 'Preview' | 'Recap' | 'Commercial';
    startTicks: number;
    endTicks: number;
}

```

4. API SERVICE (Chopper → Axios)

```

// services/jellyfin-api.ts
import axios, { AxiosInstance } from 'axios';

class JellyfinApiService {
    private client: AxiosInstance;
    private baseUrl: string = '';
    private token: string = '';
    private userId: string = '';

    constructor() {
        this.client = axios.create({
            timeout: 30000,
            headers: {
                'Content-Type': 'application/json',
                'Accept': 'application/json',
            }
        });
    }

    // Interceptor pour ajouter le token
    this.client.interceptors.request.use((config) => {
        if (this.token) {
            config.headers['X-Emby-Token'] = this.token;
        }
        return config;
    });
}

```

```

setServer(url: string) { this.baseUrl = url.replace(/\/$/, ''); }
setToken(token: string) { this.token = token; }
setUserId(id: string) { this.userId = id; }

// ===== AUTH =====
async getPublicUsers(): Promise<any[]> {
    return (await this.client.get(`${this.baseUrl}/Users/Public`)).data;
}

async authenticateByName(username: string, password: string): Promise<any> {
    const auth = 'MediaBrowser Client="RyokyLink", Device="Browser", DeviceId="xxx", Version="1.0.0"';
    return (await this.client.post(`${this.baseUrl}/Users/AuthenticateByName`,
        { Username: username, Pw: password },
        { headers: { 'X-Emby-Authorization': auth } }
    )).data;
}

// ===== USER =====
async getUser(userId: string): Promise<any> {
    return (await this.client.get(`${this.baseUrl}/Users/${userId}`)).data;
}

async updateUserConfiguration(userId: string, config: any): Promise<void> {
    await this.client.post(`${this.baseUrl}/Users/${userId}/Configuration`, config);
}

// ===== VIEWS / LIBRARIES =====
async getViews(userId: string): Promise<any[]> {
    return (await this.client.get(`${this.baseUrl}/Users/${userId}/Views`)).data.Items || [];
}

// ===== ITEMS =====
async getItems(params: {
    userId: string;
    parentId?: string;
    includeItemTypes?: string[];
    recursive?: boolean;
    sortBy?: string;
    sortOrder?: 'Ascending' | 'Descending';
    filters?: string;
    limit?: number;
    startIndex?: number;
    fields?: string[];
    imageTypeLimit?: number;
    enableImageTypes?: string[];
}): Promise<any> {
    const query = new URLSearchParams();
    query.append('UserId', params.userId);
    if (params.parentId) query.append('ParentId', params.parentId);
    if (params.includeItemTypes) query.append('IncludeItemTypes', params.includeItemTypes.join(','));
}

```

```

    if (params.recursive !== undefined) query.append('Recursive', String(params.recursive));
    if (params.sortBy) query.append('SortBy', params.sortBy);
    if (params.sortOrder) query.append('SortOrder', params.sortOrder);
    if (params.limit) query.append('Limit', String(params.limit));
    if (params.startIndex) query.append('StartIndex', String(params.startIndex));
    if (params.fields) query.append('Fields', params.fields.join(','));

    return (await this.client.get(`${this.baseUrl}/Items?${query.toString()}`)).data;
}

async getItem(userId: string, itemId: string): Promise<any> {
    return (await this.client.get(
        `${this.baseUrl}/Users/${userId}/Items/${itemId}?
        ↵ Fields=PrimaryImageAspectRatio,CanDelete,MediaSourceCount`
    )).data;
}

// ===== DASHBOARD =====
async getResumeItems(userId: string, mediaType: 'Video' | 'Audio'): Promise<any[]> {
    const data = await this.getItems({
        userId,
        mediaTypes: [mediaType],
        filters: 'IsResumable',
        sortBy: 'DatePlayed',
        sortOrder: 'Descending',
        limit: 20,
        recursive: true,
        fields: ['PrimaryImageAspectRatio', 'BasicSyncInfo', 'CanDelete', 'MediaSourceCount'],
        imageTypeLimit: 1,
        enableImageTypes: ['Primary', 'Backdrop', 'Thumb'],
    });
    return data.Items || [];
}

async getNextUp(userId: string, params?: any): Promise<any[]> {
    const query = new URLSearchParams();
    query.append('UserId', userId);
    query.append('Limit', String(params?.limit || 24));
    query.append('Fields', 'PrimaryImageAspectRatio,DateCreated,MediaSourceCount,BasicSyncInfo');
    if (params?.seriesId) query.append('SeriesId', params.seriesId);
    if (params?.parentId) query.append('ParentId', params.parentId);

    return (await this.client.get(`${this.baseUrl}/Shows/NextUp?${query.toString()}`)).data.Items || [];
}

async getRecentlyAdded(userId: string, parentId: string, limit: number = 16): Promise<any[]> {
    const data = await this.getItems({
        userId,
        parentId,
        sortBy: 'DateCreated',
        sortOrder: 'Descending',
    });

```



```

        limit,
        recursive: true,
        fields: ['PrimaryImageAspectRatio', 'BasicSyncInfo', 'CanDelete', 'MediaSourceCount'],
        imageTypeLimit: 1,
        enableImageTypes: ['Primary', 'Backdrop', 'Thumb'],
    });
    return data.Items || [];
}

// ===== PLAYBACK =====
async getPlaybackInfo(itemId: string, userId: string, params?: any): Promise<any> {
    return (await this.client.post(
        `${this.baseUrl}/Items/${itemId}/PlaybackInfo?UserId=${userId}`,
        params || {}
    )).data;
}

async reportPlaybackProgress(data: {
    itemId: string;
    positionTicks: number;
    isPaused: boolean;
    mediaSourceId?: string;
    playSessionId?: string;
}): Promise<void> {
    await this.client.post(`${this.baseUrl}/Sessions/Playing/Progress`, {
        ItemId: data.itemId,
        PositionTicks: data.positionTicks,
        IsPaused: data.isPaused,
        MediaSourceId: data.mediaSourceId,
        PlaySessionId: data.playSessionId,
    });
}

async reportPlaybackStopped(data: {
    itemId: string;
    positionTicks: number;
    mediaSourceId?: string;
    playSessionId?: string;
}): Promise<void> {
    await this.client.post(`${this.baseUrl}/Sessions/Playing/Stopped`, {
        ItemId: data.itemId,
        PositionTicks: data.positionTicks,
        MediaSourceId: data.mediaSourceId,
        PlaySessionId: data.playSessionId,
    });
}

// ===== IMAGES =====
getImageUrl(itemId: string, imageType: string = 'Primary', options?: {
    width?: number;
    height?: number;

```

```

    quality?: number;
    tag?: string;
  }): string {
    let url = `${this.baseUrl}/Items/${itemId}/Images/${imageType}`;
    const params = new URLSearchParams();
    if (options?.width) params.append('MaxWidth', String(options.width));
    if (options?.height) params.append('MaxHeight', String(options.height));
    if (options?.quality) params.append('Quality', String(options.quality));
    if (options?.tag) params.append('tag', options.tag);
    if (params.toString()) url += `?${params.toString()}`;
    return url;
  }

  // ===== SEARCH =====
  async search(userId: string, searchTerm: string, limit: number = 24): Promise<any[]> {
    const query = new URLSearchParams();
    query.append('UserId', userId);
    query.append('searchTerm', searchTerm);
    query.append('Limit', String(limit));
    query.append('Fields', 'PrimaryImageAspectRatio,CanDelete,MediaSourceCount,BasicSyncInfo');
    query.append('ImageTypeLimit', '1');
    query.append('EnableImageTypes', 'Primary,Backdrop,Thumb');

    return (await this.client.get(`${this.baseUrl}/Items?${query.toString()}`)).data.Items || [];
  }

  // ===== FAVOURITES =====
  async toggleFavorite(userId: string, itemId: string, isFavorite: boolean): Promise<void> {
    const endpoint = isFavorite ? 'FavoriteItems' : 'FavoriteItems/Delete';
    await this.client.post(`${this.baseUrl}/Users/${userId}/${endpoint}/${itemId}`);
  }

  // ===== MEDIA SEGMENTS =====
  async getMediaSegments(itemId: string): Promise<MediaSegment[]> {
    try {
      const data = await this.client.get(`${this.baseUrl}/Items/${itemId}/MediaSegments`);
      return data.data.Items || [];
    } catch {
      return [];
    }
  }
}

export const jellyfinApi = new JellyfinApiService();

```

5. STORES PINIA (exemples clés)

```

// stores/auth.ts
import { defineStore } from 'pinia';
import { ref, computed } from 'vue';
import { jellyfinApi } from '@/services/jellyfin-api';

```

```

export const useAuthStore = defineStore('auth', () => {
  // State
  const serverUrl = ref(localStorage.getItem('jellyfin_server') || '');
  const accessToken = ref(localStorage.getItem('jellyfin_token') || '');
  const userId = ref(localStorage.getItem('jellyfin_userid') || '');
  const isAuthenticated = computed(() => !!accessToken.value && !!serverUrl.value);

  // Actions
  async function login(server: string, username: string, password: string) {
    jellyfinApi.setServer(server);
    const result = await jellyfinApi.authenticateByName(username, password);

    accessToken.value = result.AccessToken;
    userId.value = result.User.Id;
    serverUrl.value = server;

    localStorage.setItem('jellyfin_server', server);
    localStorage.setItem('jellyfin_token', result.AccessToken);
    localStorage.setItem('jellyfin_userid', result.User.Id);

    jellyfinApi.setToken(result.AccessToken);
    jellyfinApi.setUserId(result.User.Id);

    return result;
  }

  function logout() {
    accessToken.value = '';
    userId.value = '';
    localStorage.removeItem('jellyfin_token');
    localStorage.removeItem('jellyfin_userid');
  }

  function restoreSession() {
    if (accessToken.value && serverUrl.value) {
      jellyfinApi.setServer(serverUrl.value);
      jellyfinApi.setToken(accessToken.value);
      jellyfinApi.setUserId(userId.value);
    }
  }

  return { serverUrl, accessToken, userId, isAuthenticated, login, logout, restoreSession };
});

```

```

// stores/dashboard.ts
import { defineStore } from 'pinia';
import { ref, computed } from 'vue';
import { jellyfinApi } from '@/services/jellyfin-api';
import { useAuthStore } from './auth';

```

```

export const useDashboardStore = defineStore('dashboard', () => {
  const auth = useAuthStore();

  // State
  const resumeVideo = ref<any[]>([]);
  const resumeAudio = ref<any[]>([]);
  const resumeBooks = ref<any[]>([]);
  const nextUp = ref<any[]>([]);
  const recentlyAdded = ref<Map<string, any[]>>(new Map());
  const isLoading = ref(false);
  const error = ref<string | null>(null);

  // Getters
  const allResume = computed(() => [
    ...resumeVideo.value,
    ...resumeAudio.value,
    ...resumeBooks.value,
  ]);

  // Actions
  async function fetchDashboard() {
    if (!auth.userId) return;
    isLoading.value = true;
    error.value = null;

    try {
      // Paralléliser les calls
      const [video, audio, books, next] = await Promise.all([
        jellyfinApi.getResumeItems(auth.userId, 'Video'),
        jellyfinApi.getResumeItems(auth.userId, 'Audio'),
        jellyfinApi.getResumeItems(auth.userId, 'Book'),
        jellyfinApi.getNextUp(auth.userId),
      ]);

      resumeVideo.value = video;
      resumeAudio.value = audio;
      resumeBooks.value = books;
      nextUp.value = next;
    } catch (e) {
      error.value = String(e);
    } finally {
      isLoading.value = false;
    }
  }

  async function fetchRecentlyAddedForView(viewId: string, viewName: string) {
    if (!auth.userId) return;
    const items = await jellyfinApi.getRecentlyAdded(auth.userId, viewId);
    recentlyAdded.value.set(viewId, { name: viewName, items });
  }
}

```

```

return {
  resumeVideo, resumeAudio, resumeBooks, nextUp,
  recentlyAdded, isLoading, error, allResume,
  fetchDashboard, fetchRecentlyAddedForView,
};
});

```

```

// stores/playback.ts
import { defineStore } from 'pinia';
import { ref, computed } from 'vue';
import { jellyfinApi } from '@/services/jellyfin-api';
import type { MediaSegment, MediaStream, MediaSourceInfo } from '@/types';

export const usePlaybackStore = defineStore('playback', () => {
  // State
  const currentItem = ref<any>(null);
  const mediaSource = ref<MediaSourceInfo | null>(null);
  const mediaStreams = ref<MediaStream[]>([]);
  const mediaSegments = ref<MediaSegment[]>([]);
  const currentAudioStreamIndex = ref<number>(-1);
  const currentSubStreamIndex = ref<number>(-1);
  const playSessionId = ref('');

  // Player state
  const isPlaying = ref(false);
  const isPaused = ref(false);
  const isBuffering = ref(false);
  const currentTime = ref(0); // seconds
  const duration = ref(0); // seconds
  const volume = ref(100);
  const playbackRate = ref(1.0);
  const isFullscreen = ref(false);
  const skippedSegments = ref<Set<string>>(new Set());

  // Getters
  const currentAudioStream = computed(() =>
    mediaStreams.value.find(s => s.type === 'Audio' && s.index === currentAudioStreamIndex.value)
  );
  const currentSubStream = computed(() =>
    mediaStreams.value.find(s => s.type === 'Subtitle' && s.index === currentSubStreamIndex.value)
  );
  const videoUrl = computed(() => {
    if (!mediaSource.value || !currentItem.value) return '';
    // Direct play
    if (mediaSource.value.supportsDirectPlay || mediaSource.value.supportsDirectStream) {
      return `${jellyfinApi.baseUrl}/Videos/${currentItem.value.id}/stream?` +
        `Static=true&MediaSourceId=${mediaSource.value.id}&` +
        `api_key=${jellyfinApi.token}&Tag=${mediaSource.value.eTag}`;
    }
    // Transcode
    if (mediaSource.value.transcodingUrl) {

```

```

        return `${jellyfinApi.baseUrl}${mediaSource.value.transcodingUrl}`;
    }
    return '';
});

const currentSegment = computed(() => {
    const posTicks = currentTime.value * 10_000_000; // seconds to ticks
    return mediaSegments.value.find(s => posTicks >= s.startTicks && posTicks < s.endTicks);
});

// Actions
async function loadItem(item: any) {
    currentItem.value = item;
    const auth = useAuthStore();

    // Get playback info
    const playbackInfo = await jellyfinApi.getPlaybackInfo(item.id, auth.userId, {
        MaxStreamingBitrate: 140000000,
        MediaSourceId: item.id,
        AutoOpenLiveStream: true,
    });

    mediaSource.value = playbackInfo.MediaSources?.[0] || null;
    mediaStreams.value = mediaSource.value?.mediaStreams || [];
    playSessionId.value = playbackInfo.PlaySessionId || '';

    // Set default streams
    const defaultAudio = mediaStreams.value.find(s => s.type === 'Audio' && s.isDefault);
    const defaultSub = mediaStreams.value.find(s => s.type === 'Subtitle' && s.isDefault);
    currentAudioStreamIndex.value = defaultAudio?.index ?? -1;
    currentSubStreamIndex.value = defaultSub?.index ?? -1;

    // Load media segments
    mediaSegments.value = await jellyfinApi.getMediaSegments(item.id);
    skippedSegments.value.clear();

    currentTime.value = 0;
    isPlaying.value = true;
    isPaused.value = false;
}

function play() { isPlaying.value = true; isPaused.value = false; }
function pause() { isPlaying.value = false; isPaused.value = true; }
function togglePlayPause() { isPlaying.value ? pause() : play(); }

function seek(seconds: number) {
    currentTime.value = Math.max(0, Math.min(seconds, duration.value));
}

function seekRelative(seconds: number) {
    seek(currentTime.value + seconds);
}

```

```

function setVolume(vol: number) { volume.value = Math.max(0, Math.min(100, vol)); }
function setPlaybackRate(rate: number) { playbackRate.value = rate; }
function toggleFullscreen() { isFullscreen.value = !isFullscreen.value; }

function skipSegment(segment: MediaSegment) {
  seek(segment.endTicks / 10_000_000);
  skippedSegments.value.add(`${segment.type}_${segment.startTicks}`);
}

async function reportProgress() {
  if (!currentItem.value) return;
  await jellyfinApi.reportPlaybackProgress({
    itemId: currentItem.value.id,
    positionTicks: Math.floor(currentTime.value * 10_000_000),
    isPaused: isPaused.value,
    mediaSourceId: mediaSource.value?.id,
    playSessionId: playSessionId.value,
  });
}

async function reportStopped() {
  if (!currentItem.value) return;
  await jellyfinApi.reportPlaybackStopped({
    itemId: currentItem.value.id,
    positionTicks: Math.floor(currentTime.value * 10_000_000),
    mediaSourceId: mediaSource.value?.id,
    playSessionId: playSessionId.value,
  });
}

return {
  currentItem, mediaSource, mediaStreams, mediaSegments,
  currentAudioStreamIndex, currentSubStreamIndex, playSessionId,
  isPlaying, isPaused, isBuffering, currentTime, duration,
  volume, playbackRate, isFullscreen, skippedSegments,
  currentAudioStream, currentSubStream, videoUrl, currentSegment,
  loadItem, play, pause, togglePlayPause, seek, seekRelative,
  setVolume, setPlaybackRate, toggleFullscreen, skipSegment,
  reportProgress, reportStopped,
};
});

```

6. COMPOSANTS VUE (exemples)

```

<!-- components/media/PosterRow.vue -->
<template>
  <div class="poster-row">
    <div class="row-header" v-if="label">
      <h3 class="row-label">{{ label }}</h3>
      <button v-if="onLabelClick" class="see-all" @click="onLabelClick">

```

```

        Voir tout →
      </button>
    </div>
    <div class="row-scroll" ref="scrollContainer">
      <PosterCard
        v-for="item in posters"
        :key="item.id"
        :item="item"
        :aspect-ratio="aspectRatio"
        @select="$emit('select', item)"
        @hover="$emit('hover', item)"
      />
    </div>
  </div>
</template>

<script setup lang="ts">
import { ref } from 'vue';
import PosterCard from './PosterCard.vue';

interface Props {
  label?: string;
  posters: any[];
  aspectRatio?: number;
  onLabelClick?: () => void;
}

defineProps<Props>();
defineEmits(['select', 'hover']);

const scrollContainer = ref<HTMLElement>();
</script>

<style scoped>
.poster-row {
  margin-bottom: 24px;
}

.row-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 16px;
  margin-bottom: 12px;
}

.row-label {
  font-size: 1.2rem;
  font-weight: 600;
  color: var(--sao-text);
  margin: 0;
}

.see-all {

```



```

background: none;
border: none;
color: var(--sao-cyan);
cursor: pointer;
font-size: 0.9rem;
}
.row-scroll {
display: flex;
gap: 12px;
overflow-x: auto;
scroll-snap-type: x mandatory;
padding: 0 16px;
scrollbar-width: none;
}
.row-scroll::-webkit-scrollbar {
display: none;
}
</style>

```

```

<!-- components/media/PosterCard.vue -->
<template>
  <div
    class="poster-card"
    :style="cardStyle"
    @click="$emit('select', item)"
    @mouseenter="$emit('hover', item)"
  >
    <div class="poster-image-wrapper">
      
      <div v-if="progress > 0" class="progress-bar">
        <div class="progress-fill" :style="{ width: `${progress}%` }" />
      </div>
      <div v-if="item.userData?.isFavorite" class="favorite-badge">
        ⇨ $${vcenter}{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/doc/miner/emoji/2665.png}}}}}}$
      </div>
    </div>
    <div class="poster-info">
      <span class="poster-title">{{ item.name }}</span>
      <span v-if="item.productionYear" class="poster-year">{{ item.productionYear }}</span>
    </div>
  </div>
</template>

<script setup lang="ts">
import { computed } from 'vue';

```

```

import { useImageUrl } from '@composables/useImageUrl';

interface Props {
  item: any;
  aspectRatio?: number;
}

const props = defineProps<Props>();
defineEmits(['select', 'hover']);

const { getPrimaryImageUrl } = useImageUrl();

const imageUrl = computed(() => getPrimaryImageUrl(props.item.id, {
  width: 300,
  tag: props.item.imageTags?.Primary,
}));

const progress = computed(() => {
  if (!props.item.userData?.playbackPositionTicks || !props.item.runTimeTicks) return 0;
  return (props.item.userData.playbackPositionTicks / props.item.runTimeTicks) * 100;
});

const cardStyle = computed(() => ({
  aspectRatio: props.aspectRatio ? `${props.aspectRatio}` : '2/3',
}));
</script>

<style scoped>
.poster-card {
  flex-shrink: 0;
  width: 160px;
  cursor: pointer;
  transition: transform 0.2s ease;
  scroll-snap-align: start;
}
.poster-card:hover {
  transform: scale(1.05);
}
.poster-image-wrapper {
  position: relative;
  border-radius: 8px;
  overflow: hidden;
  background: var(--sao-surface);
}
.poster-image {
  width: 100%;
  height: 100%;
  object-fit: cover;
  display: block;
}
.progress-bar {

```

```

    position: absolute;
    bottom: 0;
    left: 0;
    right: 0;
    height: 3px;
    background: rgba(255,255,255,0.2);
  }
  .progress-fill {
    height: 100%;
    background: var(--sao-cyan);
    transition: width 0.3s;
  }
  .favorite-badge {
    position: absolute;
    top: 8px;
    right: 8px;
    color: #ff4444;
    font-size: 1.2rem;
  }
  .poster-info {
    padding: 8px 4px;
  }
  .poster-title {
    display: block;
    font-size: 0.9rem;
    font-weight: 500;
    color: var(--sao-text);
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
  }
  .poster-year {
    font-size: 0.8rem;
    color: var(--sao-dim);
  }
</style>

```

```

<!-- views/DashboardView.vue -->
<template>
  <div class="dashboard">
    <!-- Hero Banner -->
    <MediaBanner
      v-if="carouselItems.length > 0"
      :items="carouselItems"
      @select="goToItem"
    />

    <!-- Poster Rows -->
    <div class="dashboard-content">
      <PosterRow
        v-if="dashboardStore.resumeVideo.length > 0"

```

```

        label="Continuer à regarder"
        :posters="dashboardStore.resumeVideo"
        @select="goToItem"
    />

    <PosterRow
        v-if="dashboardStore.nextUp.length > 0"
        label="Prochain épisode"
        :posters="dashboardStore.nextUp"
        @select="goToItem"
    />

    <PosterRow
        v-for="[viewId, viewData] in dashboardStore.recentlyAdded"
        :key="viewId"
        :label="`Ajoutés récemment dans ${viewData.name}`"
        :posters="viewData.items"
        @select="goToItem"
    />
</div>
</div>
</template>

<script setup lang="ts">
import { computed, onMounted } from 'vue';
import { useRouter } from 'vue-router';
import { useDashboardStore } from '@/stores/dashboard';
import { useViewsStore } from '@/stores/views';
import MediaBanner from '@/components/media/MediaBanner.vue';
import PosterRow from '@/components/media/PosterRow.vue';

const router = useRouter();
const dashboardStore = useDashboardStore();
const viewsStore = useViewsStore();

const carouselItems = computed(() => {
    // Configurable: nextUp, resume, or combined
    return [...dashboardStore.allResume, ...dashboardStore.nextUp];
});

onMounted(async () => {
    await dashboardStore.fetchDashboard();
    await viewsStore.fetchViews();

    // Fetch recently added for each view
    for (const view of viewsStore.views) {
        await dashboardStore.fetchRecentlyAddedForView(view.id, view.name);
    }
});

function goToItem(item: any) {

```

```

    router.push({ name: 'item', params: { id: item.id } });
  }
</script>

```

7. COMPOSABLES

```

// composables/useImageUrl.ts
import { computed } from 'vue';
import { jellyfinApi } from '@/services/jellyfin-api';

export function useImageUrl() {
  function getPrimaryImageUrl(
    itemId: string,
    options?: { width?: number; height?: number; tag?: string }
  ): string {
    return jellyfinApi.getImageUrl(itemId, 'Primary', options);
  }

  function getBackdropUrl(
    itemId: string,
    options?: { width?: number; height?: number; tag?: string }
  ): string {
    return jellyfinApi.getImageUrl(itemId, 'Backdrop', options);
  }

  function getLogoUrl(itemId: string, tag?: string): string {
    return jellyfinApi.getImageUrl(itemId, 'Logo', { width: 400, tag });
  }

  function getThumbUrl(itemId: string, tag?: string): string {
    return jellyfinApi.getImageUrl(itemId, 'Thumb', { width: 600, tag });
  }

  return { getPrimaryImageUrl, getBackdropUrl, getLogoUrl, getThumbUrl };
}

```

```

// composables/useKeyboardShortcuts.ts
import { onMounted, onUnmounted } from 'vue';

export type VideoHotKey =
  | 'playPause' | 'volumeUp' | 'volumeDown' | 'speedUp' | 'speedDown'
  | 'fullScreen' | 'skipMediaSegment' | 'exit' | 'mute' | 'nextVideo'
  | 'prevVideo' | 'nextChapter' | 'prevChapter' | 'toggleSubtitles'
  | 'seekForward' | 'seekBack' | 'stepForward' | 'stepBack';

const KEY_MAP: Record<string, VideoHotKey> = {
  ' ': 'playPause',
  'k': 'playPause',
  'ArrowUp': 'volumeUp',
  'ArrowDown': 'volumeDown',
  'ArrowRight': 'seekForward',

```

```

    'ArrowLeft': 'seekBack',
    'f': 'fullScreen',
    'm': 'mute',
    'n': 'nextVideo',
    'p': 'prevVideo',
    's': 'toggleSubtitles',
    'Escape': 'exit',
  };

export function useKeyboardShortcuts(
  handlers: Partial<Record<VideoHotKey, () => void>>
) {
  function handleKeydown(e: KeyboardEvent) {
    const action = KEY_MAP[e.key];
    if (action && handlers[action]) {
      e.preventDefault();
      handlers[action]!();
    }
  }

  onMounted(() => window.addEventListener('keydown', handleKeydown));
  onUnmounted(() => window.removeEventListener('keydown', handleKeydown));
}

```

```

// composables/useMediaSegments.ts
import { computed, watch } from 'vue';
import { usePlaybackStore } from '@stores/playback';

export function useMediaSegments() {
  const playback = usePlaybackStore();

  const currentSegment = computed(() => playback.currentSegment);
  const shouldAutoSkip = computed(() => {
    if (!currentSegment.value) return false;
    const segment = currentSegment.value;
    const segmentId = `${segment.type}_${segment.startTicks}`;

    // Check settings for this segment type
    // const settings = useSettingsStore();
    // const skipType = settings.segmentSkipSettings[segment.type];
    // return skipType === 'skip' || (skipType === 'skipOnce' && !
    ↪ playback.skippedSegments.has(segmentId));

    return true; // Default: auto-skip everything
  });

  watch(currentSegment, (segment) => {
    if (segment && shouldAutoSkip.value) {
      playback.skipSegment(segment);
    }
  });
}

```

```
return { currentSegment, shouldAutoSkip };
}
```

📦 PACKAGE.JSON (dépendances)

```
{
  "name": "ryoky-link",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vue-tsc && vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "vue": "^3.4.0",
    "vue-router": "^4.3.0",
    "pinia": "^2.1.7",
    "axios": "^1.7.0",
    "video.js": "^8.12.0",
    "@videojs/http-streaming": "^3.10.0",
    "screenfull": "^6.0.2"
  },
  "devDependencies": {
    "@vitejs/plugin-vue": "^5.0.0",
    "typescript": "^5.4.0",
    "vue-tsc": "^2.0.0",
    "vite": "^5.2.0",
    "@types/video.js": "^7.3.58"
  }
}
```

🎨 INTEGRATION STYLE RYOKY

Les composants utilisent déjà tes variables CSS SAO :

- --sao-bg, --sao-surface, --sao-text
- --sao-cyan, --sao-amber, --sao-dim
- Le scanline, le circuit dashed, tout le bazar cyberpunk

Le player overlay utilise des gradients noirs pour les contrôles (comme Fladder) mais avec tes couleurs RYOKY pour les accents.

PLAN DE MIGRATION PAR ÉTAPES

Étape 1 : Fondations (1-2 jours)

- Créer la structure de dossiers
- Installer dépendances
- Configurer router + Pinia
- Implémenter `jellyfin-api.ts`
- Implémenter `stores/auth.ts`
- Page Login fonctionnelle

Étape 2 : Dashboard (2-3 jours)

- Types TypeScript
- `stores/dashboard.ts` + `stores/views.ts`
- `PosterCard.vue` + `PosterRow.vue`
- `MediaBanner.vue`
- `DashboardView.vue`
- Test avec ton serveur Jellyfin

Étape 3 : Détail Item (2 jours)

- `ItemDetailView.vue`
- `CastList.vue`, `RelatedItems.vue`
- Navigation item → détail

Étape 4 : Player Vidéo (3-4 jours)

- Intégration `Video.js`
- `PlayerView.vue` + `PlayerControls.vue`
- Raccourcis clavier
- Media segments (skip intro)
- Sélecteurs audio/sous-titres
- Report progress à Jellyfin

Étape 5 : Bibliothèque & Recherche (2 jours)

- `LibraryView.vue`
- Filtres et tri
- `SearchView.vue`

Étape 6 : Polish (2 jours)

- Responsive mobile/tablet/desktop
- Animations transitions
- Gestion erreurs
- Offline mode (IndexedDB)

CE QUI EST DIFFICILE / PAS PRIORITAIRE

Feature Fladder	Difficulté	Note
Sync offline	🔴 Haute	IndexedDB + background sync complexe
Multi-profile	🟡 Moyenne	Gestion multi-user dans localStorage
Live TV	🟡 Moyenne	Nécessite HLS live streaming
Comics/books	🟢 Facile	PDF/EPUB viewer
Seerr integration	🟡 Moyenne	API externe supplémentaire
Download items	🔴 Haute	FileSystem API limitée en web
Picture-in-Picture	🟡 Moyenne	API native du navigateur
Background playback	🟡 Moyenne	Media Session API

💡 CONSEILS DE CONVERSION

1. **Ne pas tout copier** : Fladder a du code généré (`.freezed.dart`, `.g.dart`) → ignorer, réécrire les interfaces TS à la main
2. **Riverpod** → **Pinia** : Les `StateNotifier` deviennent des stores Pinia avec `ref()`
3. **WidgetRef** → **composables** : Les `ref.watch()` deviennent des `computed()` ou des appels directs dans les composables
4. **BuildContext** → **Vue template** : Pas besoin de `context`, tout est réactif dans le template
5. **MediaQuery** → **CSS + composable** : `useResponsive()` pour les breakpoints, CSS pour le reste
6. **GestureDetector** → **events natifs** : `@click`, `@dblclick`, touch events, etc.

Voilà, t'as le plan complet. C'est un gros morceau mais c'est faisable étape par étape.

Tu veux que je commence à coder la **Phase 1** (fondations + login) ou tu préfères attaquer un truc spécifique ? 🔥